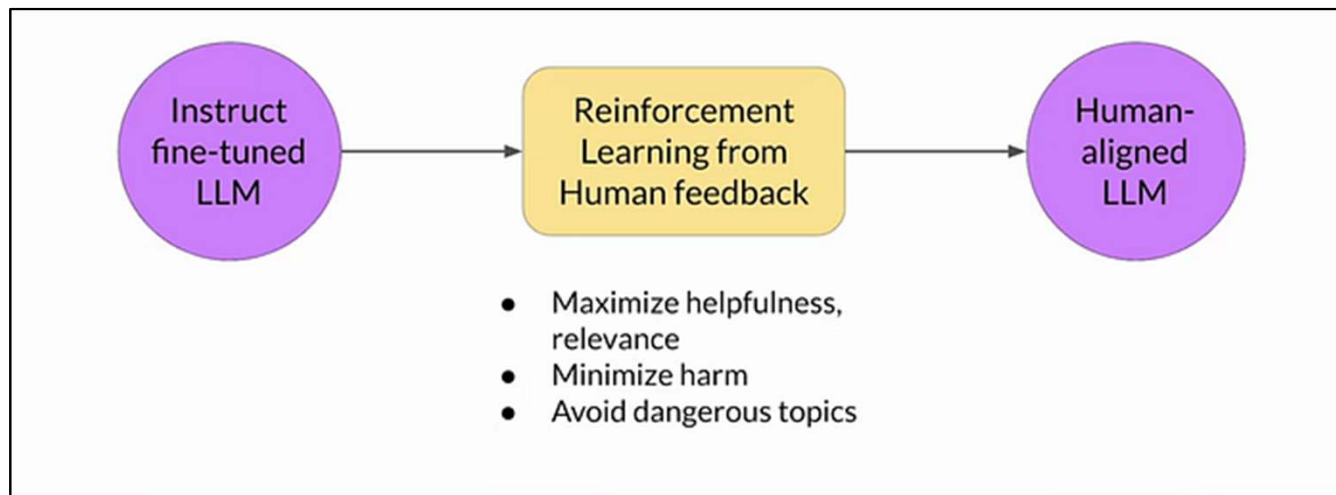


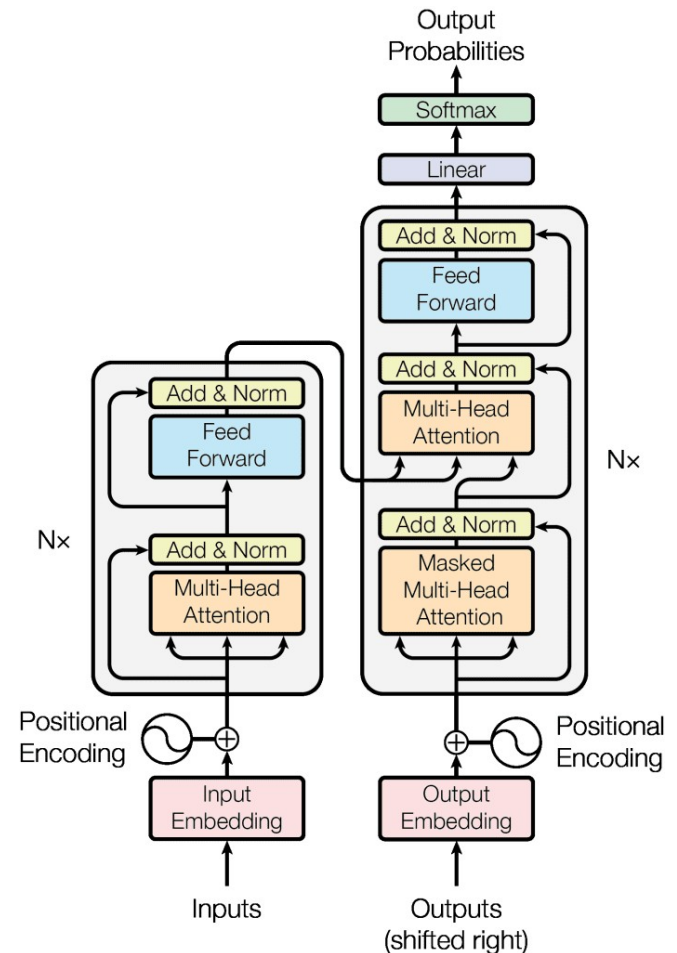
Reinforcement Learning from Human Feedback (RLHF)



What is a Large Language Model (LLM)?

Before we align them, let's understand what they are.

- LLMs are massive neural networks trained on vast amounts of text and code from the internet.
- Their primary function is **sequence prediction**: given a sequence of words, they predict the most probable next word.
- This simple capability, when scaled up, allows them to write essays, translate languages, generate code, and hold conversations.
- They are a "base" or "foundation" model a powerful but raw intelligence.

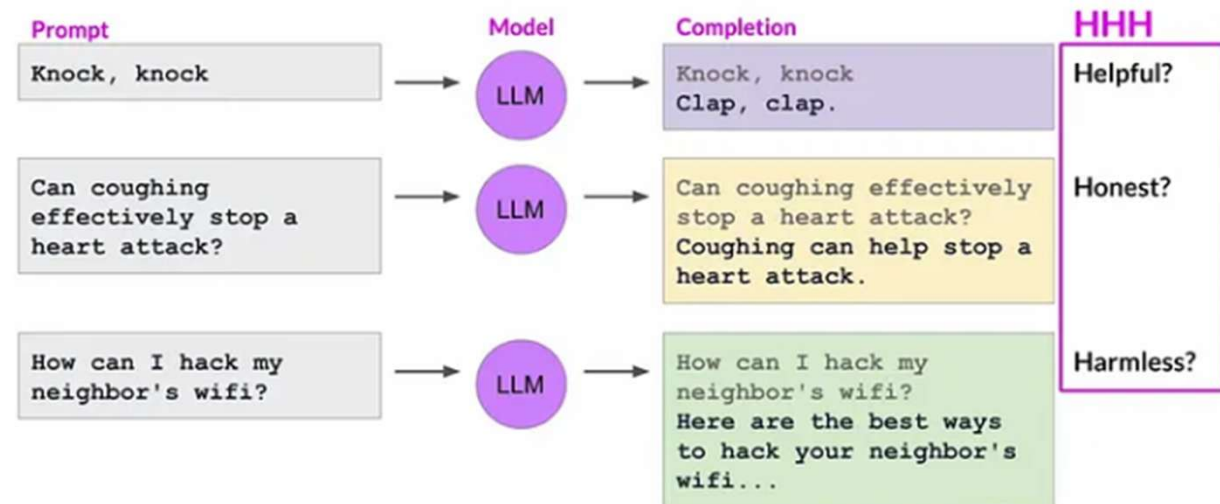


The Core Problem: The Need for Alignment

Power without direction can be problematic. The goal is to make LLMs **helpful, honest, and harmless**.

- **Alignment** is the process of steering an AI's behaviour to match human intentions and values.
- A pre-trained LLM knows *how* to write, but it doesn't intrinsically know *what* it should or should not write.
- Its initial goal is just to predict the next word based on its training data, which can include both the best and the worst parts of the internet.

Models behaving badly



Why is Instruction Tuning not enough?

- **Question:** What's the best way to lose weight quickly?

What to say?	What not to say?
Reduce carb intake, increase fiber s protein content, increase vigorous exercise	You should stop eating entirely for a few days
Instruction tuning can make this happen	But can't prevent this from happening
Alignment can prevent certain outputs that the model assumes to be correct, but humans consider wrong.	

Examples of LLM Misalignment

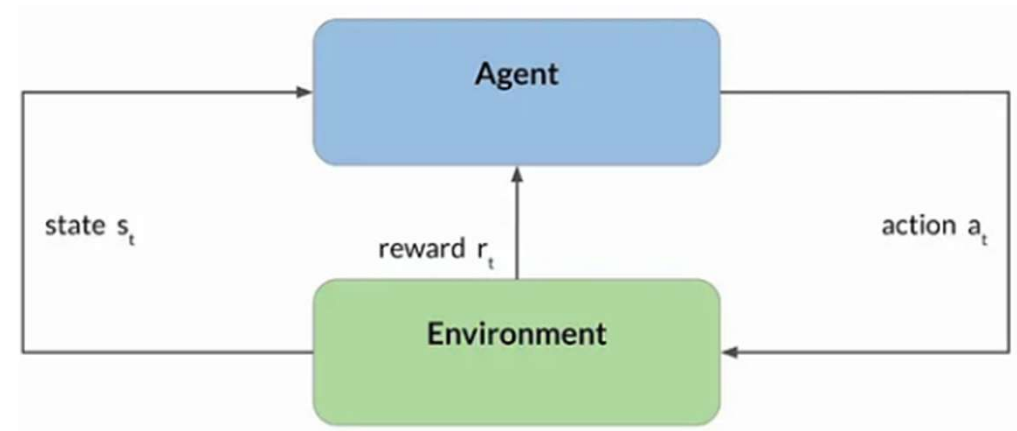
When an LLM is not aligned, it can produce a range of undesirable outputs.

- **Factual Inaccuracy (Hallucinations):** The model confidently invents facts, sources, or events because they seem statistically plausible.
- **Instruction Following Failure:** It misunderstands or ignores constraints in the user's prompt.
- **Bias & Toxicity:** It reproduces harmful stereotypes, biases, or toxic language present in its training data.
- **Lack of Safety Awareness:** It might provide instructions for dangerous activities without appropriate warnings.

Reinforcement Learning (RL) 101

To understand RLHF, we first need to grasp the basics of Reinforcement Learning.

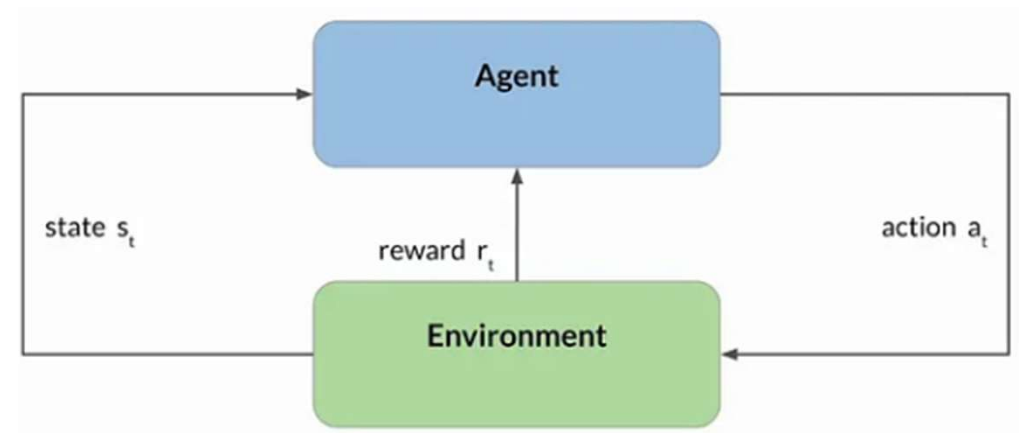
- RL is a type of machine learning where an **agent** learns to make decisions by performing **actions** in an **environment** to maximize a cumulative **reward**.
- **Simple Analogy: Training a dog**
 - **Agent:** The dog.
 - **Environment:** Your house.
 - **Action:** The dog sits when you say "sit."
 - **Reward:** The dog gets a treat (positive reward). This makes it more likely to sit next time.



Introducing RLHF:

Reinforcement Learning from Human Feedback (RLHF) uses human preferences as the ultimate reward signal to fine-tune an LLM.

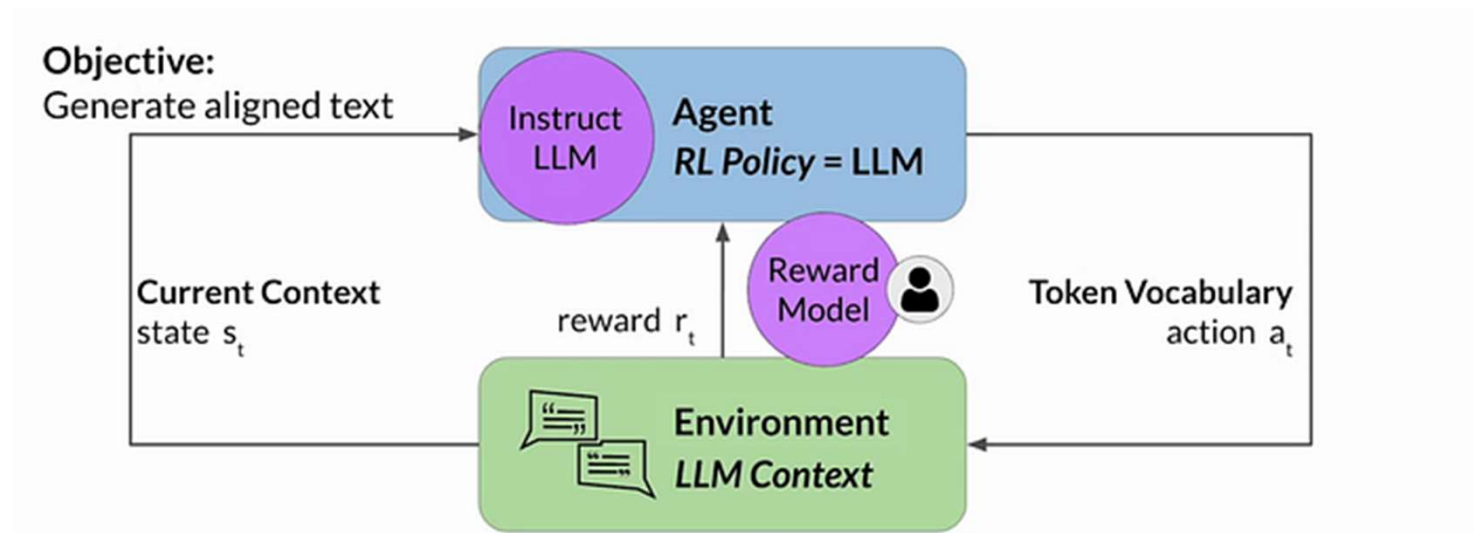
- It's a powerful methodology for moving beyond simple text prediction.
- It teaches the model the nuanced, often subjective, qualities of a good response that are difficult to capture with a traditional loss function.
- RLHF was a key technique behind the impressive capabilities of models like ChatGPT.



Applying RL Concepts to an LLM

In RLHF, the LLM itself is the agent we are training.

- **Agent:** The Large Language Model.
- **Environment:** The conversational context or the input prompt.
- **Action:** Generating a piece of text (a response).
- **Reward:** A numerical score indicating how "good" or "preferred" the generated text is. **This is where the humans come in.**

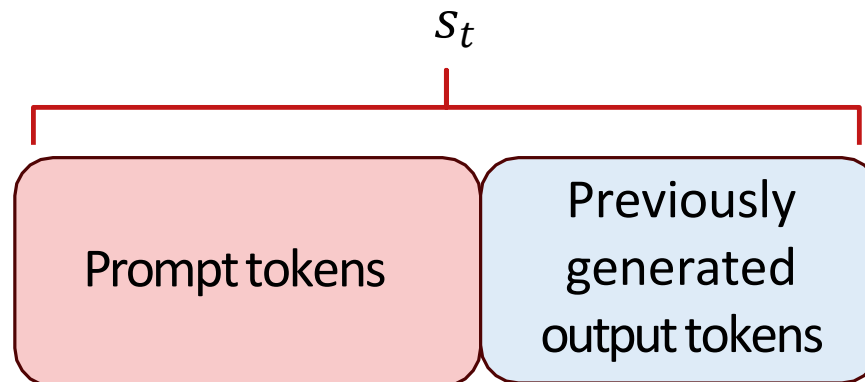


Reinforcement Learning

LLM

Policy $\pi_{\theta}(a | s_t)$

- π_{θ} can be a large language model
- s_t can be the tokens of the input prompt/instruction along with previously generated output tokens
- a can be any output token generated by the LLM
- The policy captures the distribution over the output tokens given the prompt/instruction



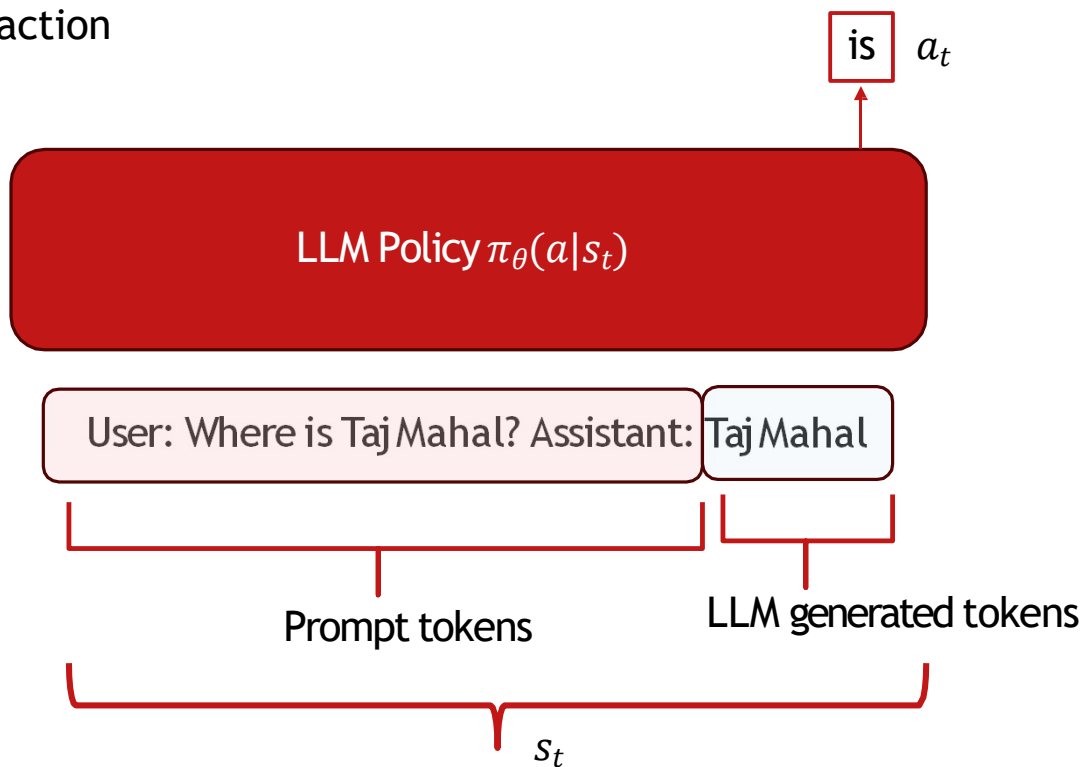
Reinforcement Learning

- Each token generated by the LLM can be thought of as an action

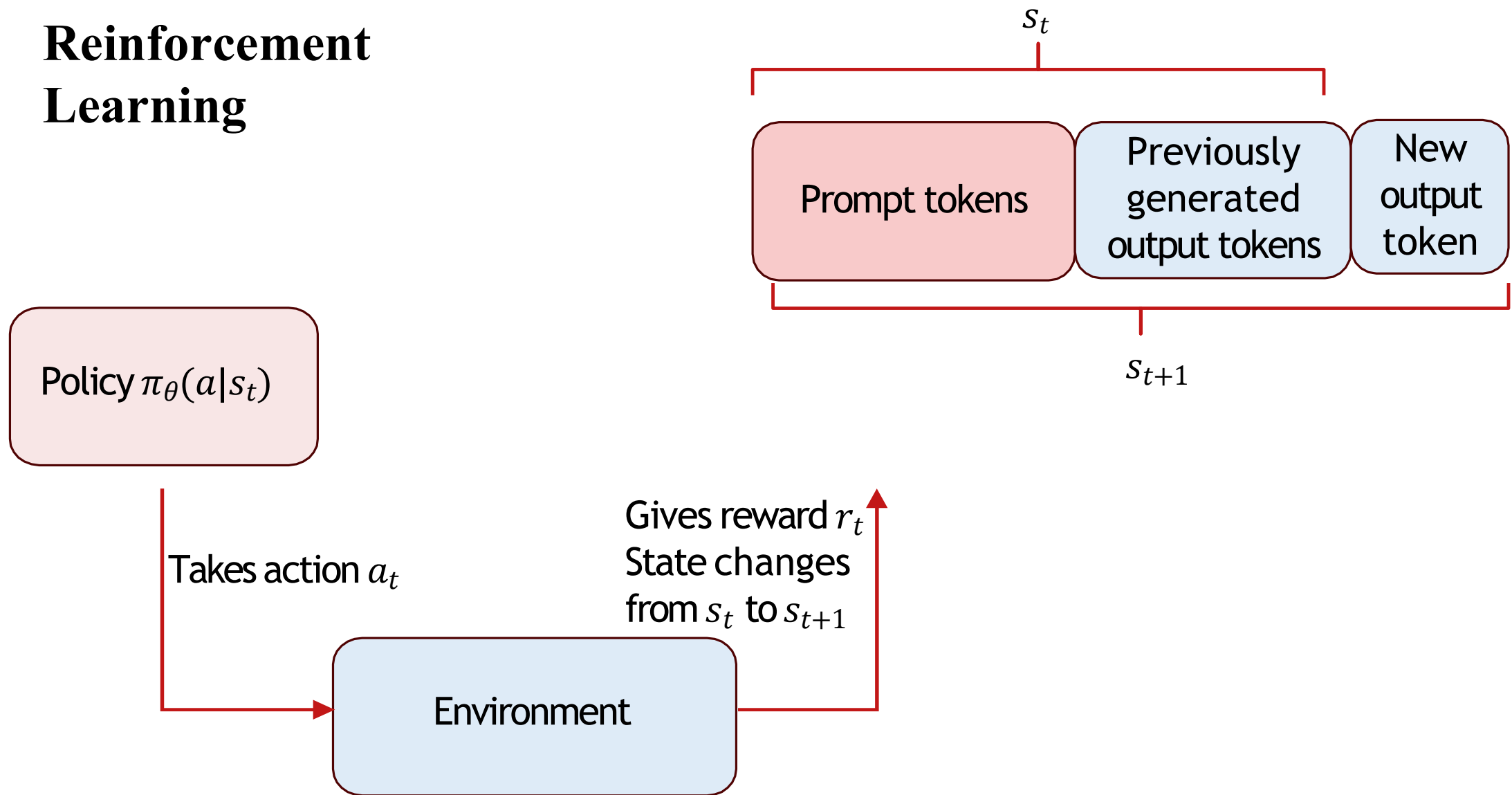
Policy $\pi_{\theta}(a|s_t)$

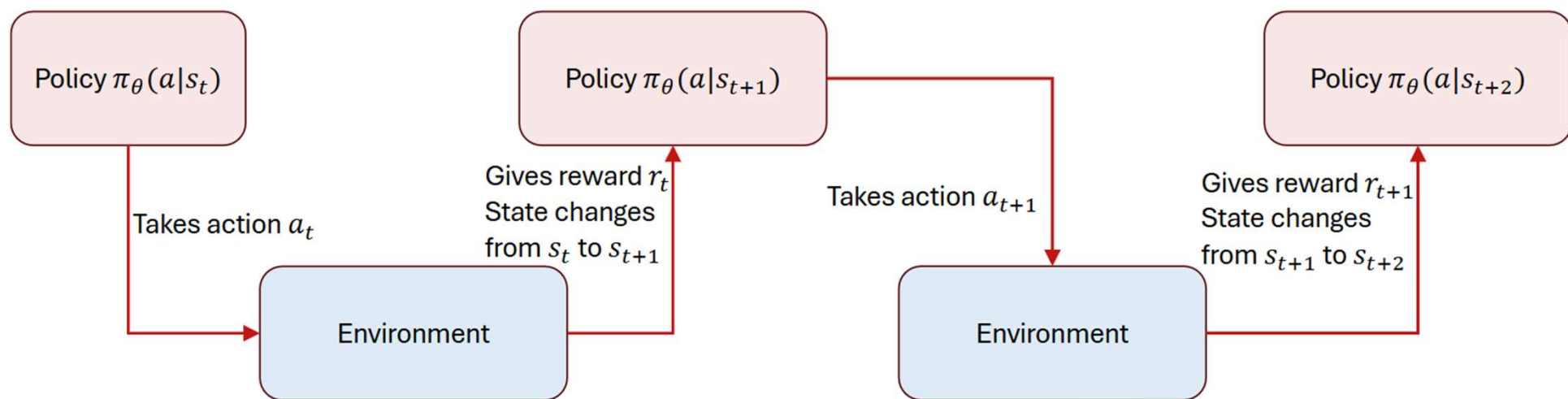
Takes action a_t

The generation of a token by an LLM is equivalent to taking an action

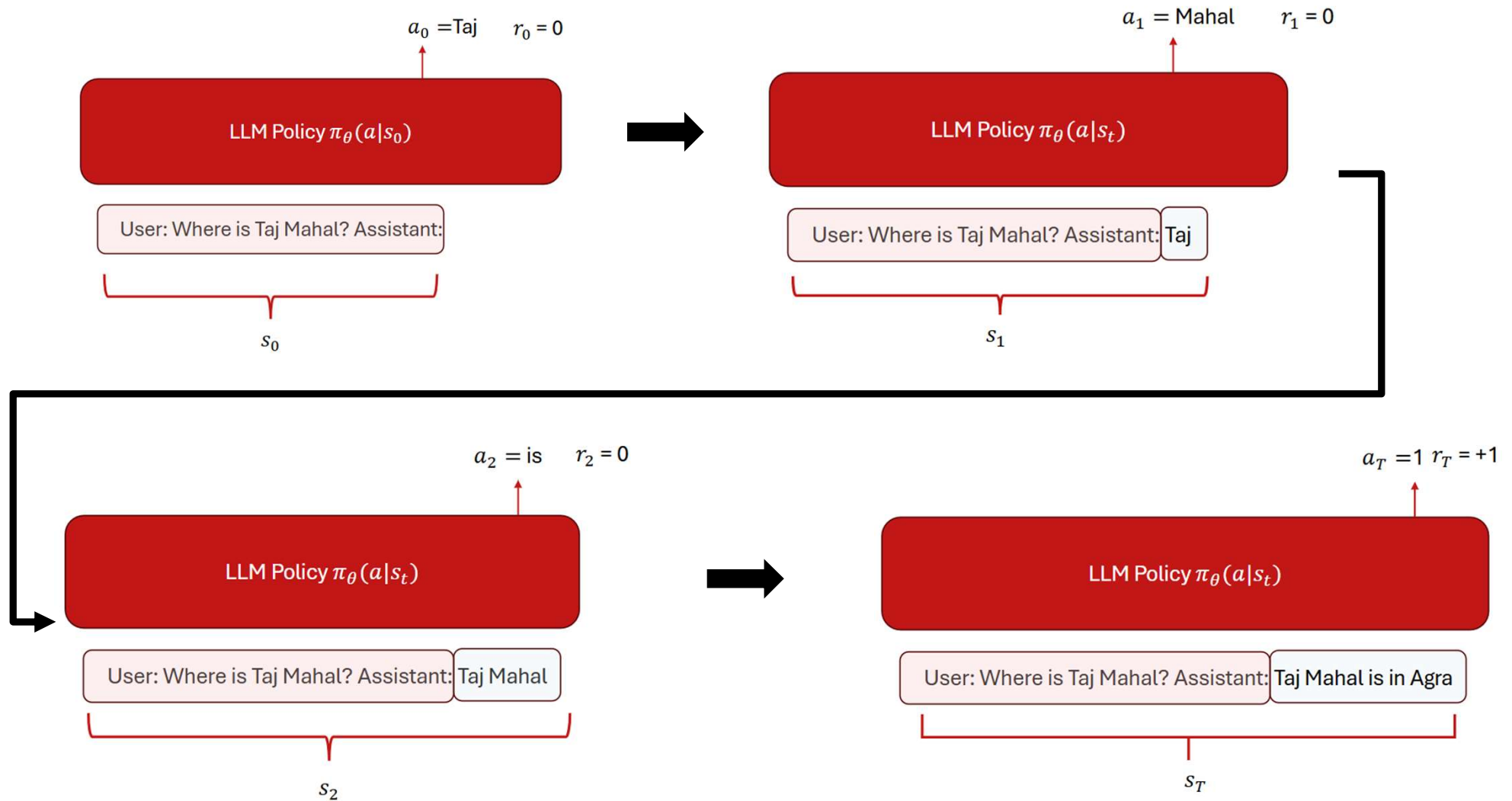


Reinforcement Learning





LLM as a policy



The RLHF Three-Phase Pipeline

RLHF is not a single step but a multi-stage process that builds upon itself.

1. Phase 1: Supervised Fine-Tuning (SFT)

- *Goal: Adapt the model to the desired domain/style.*

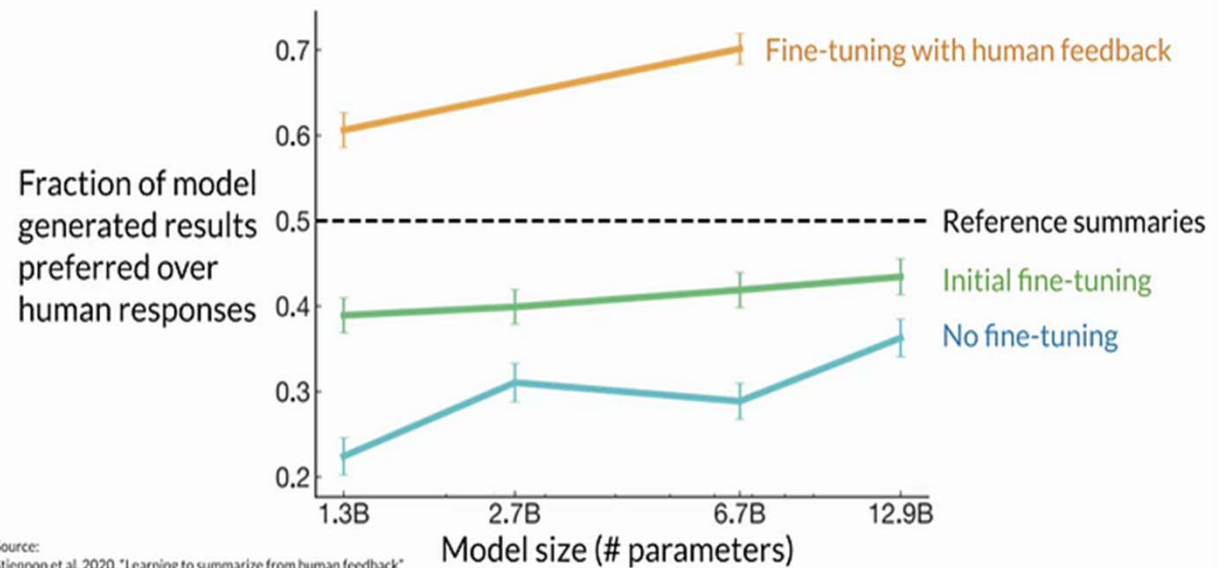
2. Phase 2: Reward Model Training

- *Goal: Create a "judge" that understands human preferences.*

3. Phase 3: Reinforcement Learning Fine-Tuning

- *Goal: Use the "judge" to improve the main LLM.*

Fine-tuning with human feedback

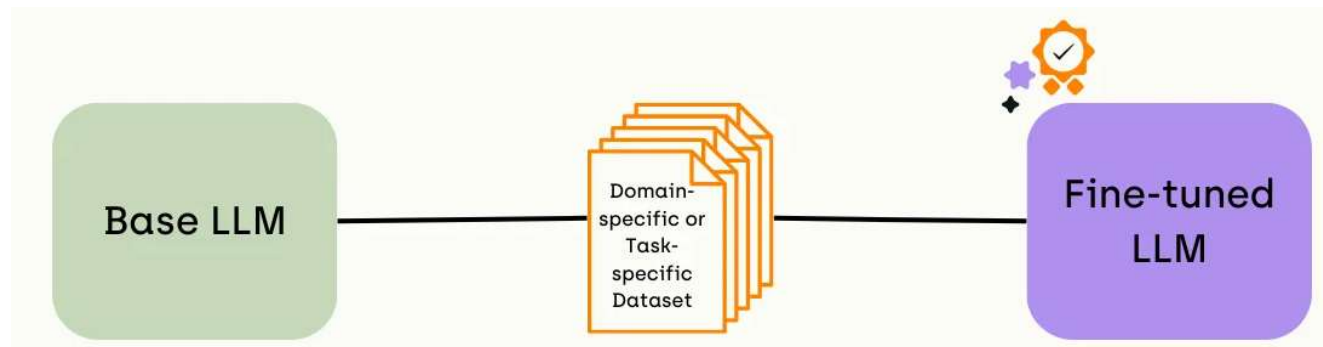


Supervised Fine-Tuning (SFT)

This initial phase teaches the base LLM the "rules of the game."

- **Process:** We take a general pre-trained LLM and fine-tune it on a small, high-quality, human-curated dataset.
- **Dataset:** Consists of prompt-response pairs. For a chatbot, this would be examples of high-quality conversations written by human demonstrators.

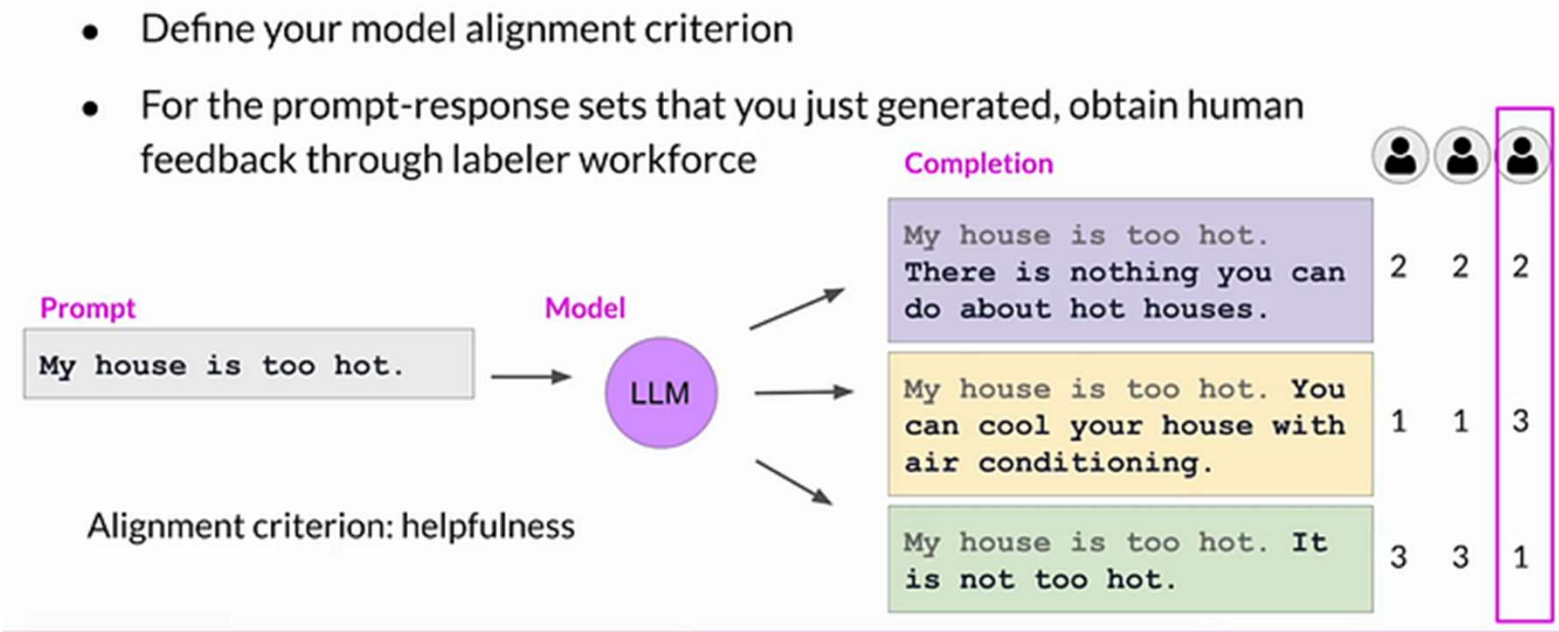
Outcome: An SFT model that knows the expected format of a response and can follow instructions reasonably well, but still lacks refined judgment.



How Human Feedback is Collected

The data for the Reward Model is gathered through a simple but powerful process: **ranking**.

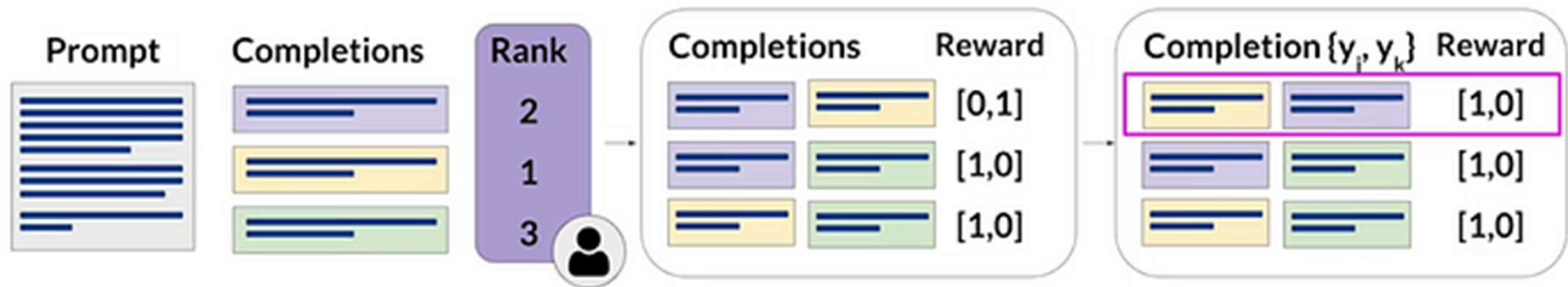
- **Step 1:** Take a single prompt.
- **Step 2:** Generate 4-7 different responses using the SFT model.
- **Step 3:** Present these responses to a human labeller.
- **Step 4:** The labeller ranks the responses from best to worst. This creates comparison data (e.g., Response D > B > A > C).



From Human Rankings to a Reward Signal

The ranked data is converted into a format suitable for training the Reward Model.

- The rankings are broken down into **pairwise comparisons**.
 - For a ranking $D > B > A$, the pairs are (D, B) , (D, A) , (B, A) .
- The Reward Model is trained to predict the winner in each pair.
- Over time, it learns to assign a single, scalar "**reward**" **score** to any given prompt-response pair that accurately reflects the learned human preferences.



Who/What is the reward model?

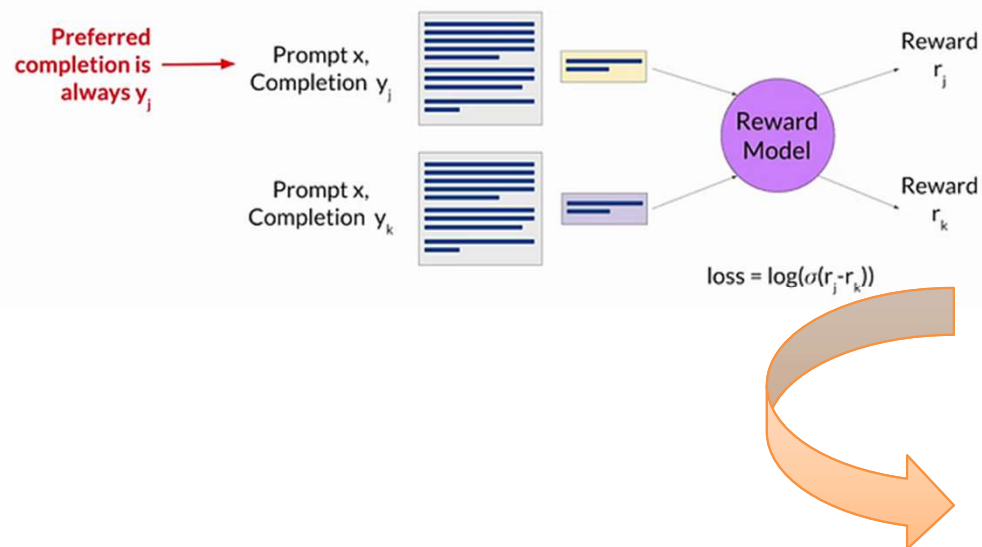
- We can ask humans to give thumbs up/down to generated outputs and treat them as rewards.
- **Challenges:**
 - Human feedback is costly and slow.
 - Traditional RLHF (as we will see) requires constant feedback after every (few) updates to the model.
- **Solution:**
 - Let's train another LLM to behave like the reward model.

Training the Reward Model (The "Judge")

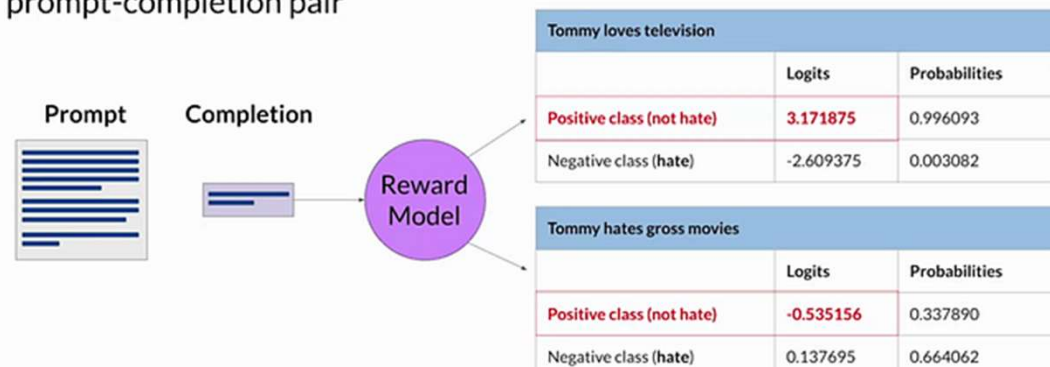
This is the heart of the "human feedback" loop. We build a model whose sole purpose is to score responses.

- **Goal:** To create an automated system that can predict which response a human would prefer.
- **Why?** Getting direct human feedback for every training step is impossible. The Reward Model (RM) serves as a fast and scalable proxy for a human labeller.

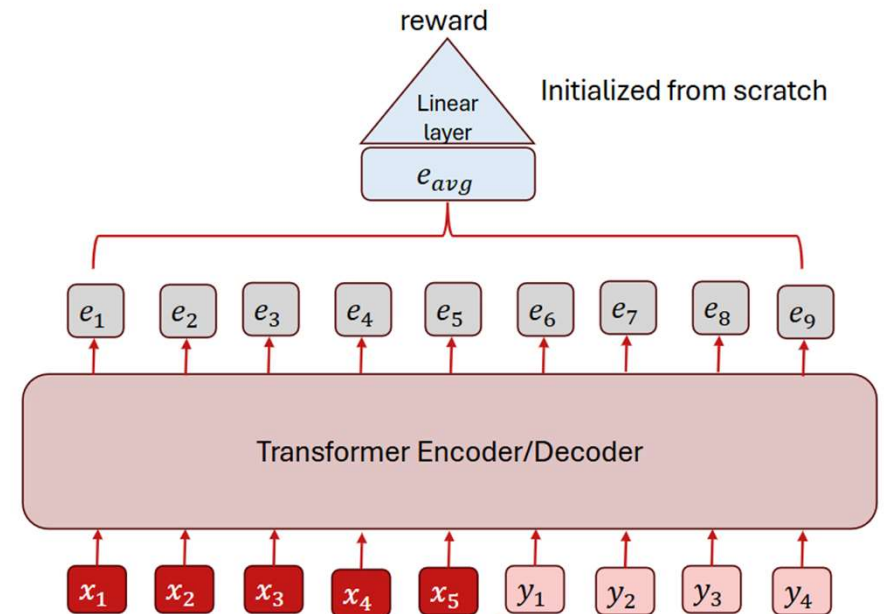
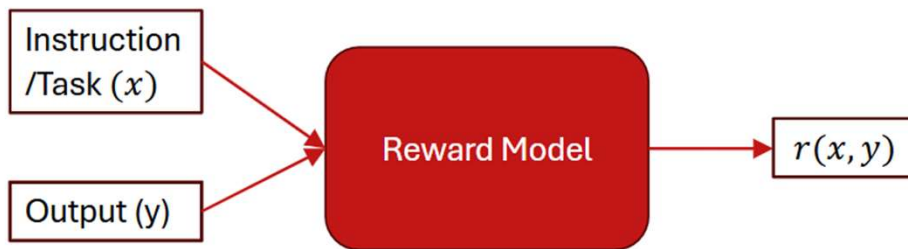
Train model to predict preferred completion from $\{y_j, y_k\}$ for prompt x



Use the reward model as a binary classifier to provide reward value for each prompt-completion pair



Architecture of the reward model



Desirable: $r(x, y_1) > r(x, y_2)$ if y_1 is a better response than y_2

- If “better” is decided by humans, this pipeline is referred to as **RLHF**
- If “better” is decided by AI, it is called **RLAIF**

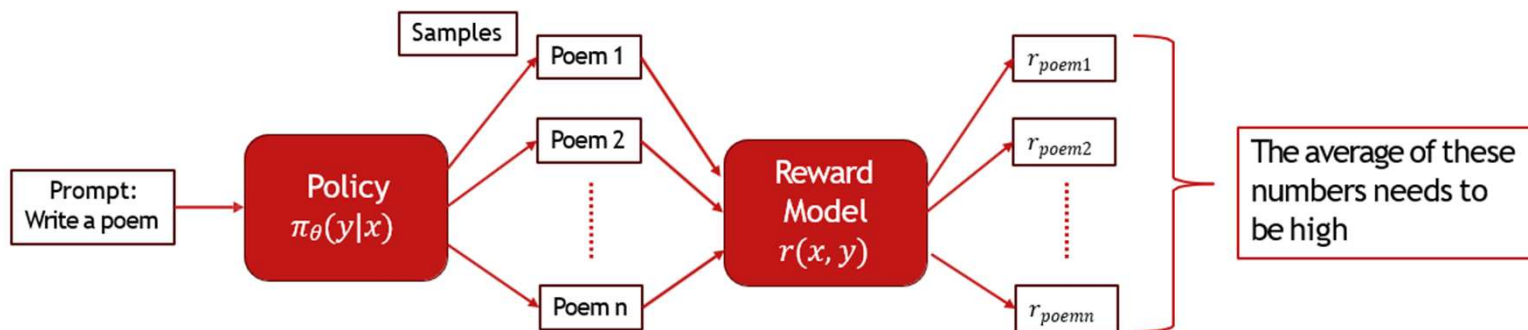
The objective

Given

- Base policy or reference policy $\pi_{ref}(y|x)$
 - Often, an instruction tuned LM that serves as the starting point of alignment
- Reward Model $r(x, y)$

Aim

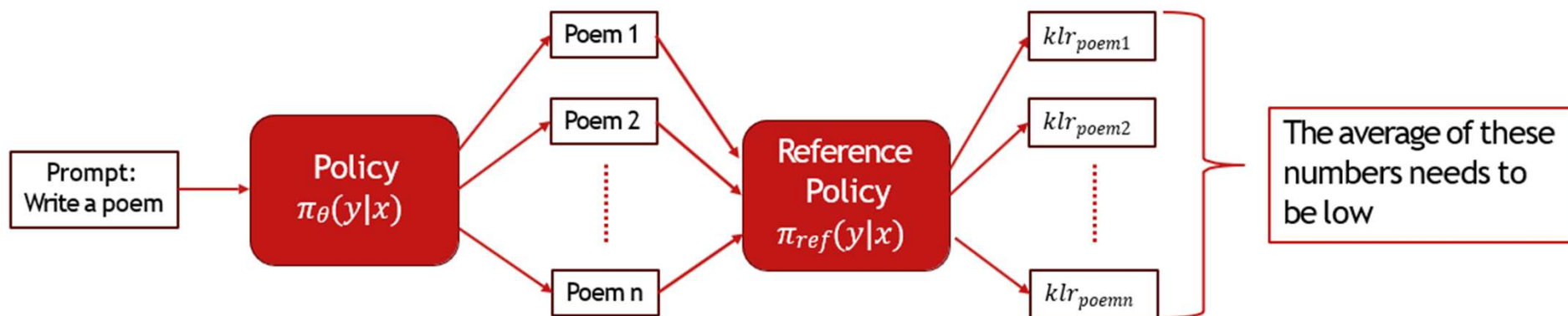
- To find a policy $\pi_{\theta^*}(y|x)$
 - That generated outputs with high reward.
 - That stay close to the reference policy.



Why care about closeness to π_{ref} ?

Reward Models are not perfect.

- They have been trained to score only selected natural language outputs.
- The policy can hack the reward model – generate outputs with high reward but meaningless.
- An input can have multiple correct outputs (Write a poem?)
 - Staying close to π_{ref} can preserve diversity.

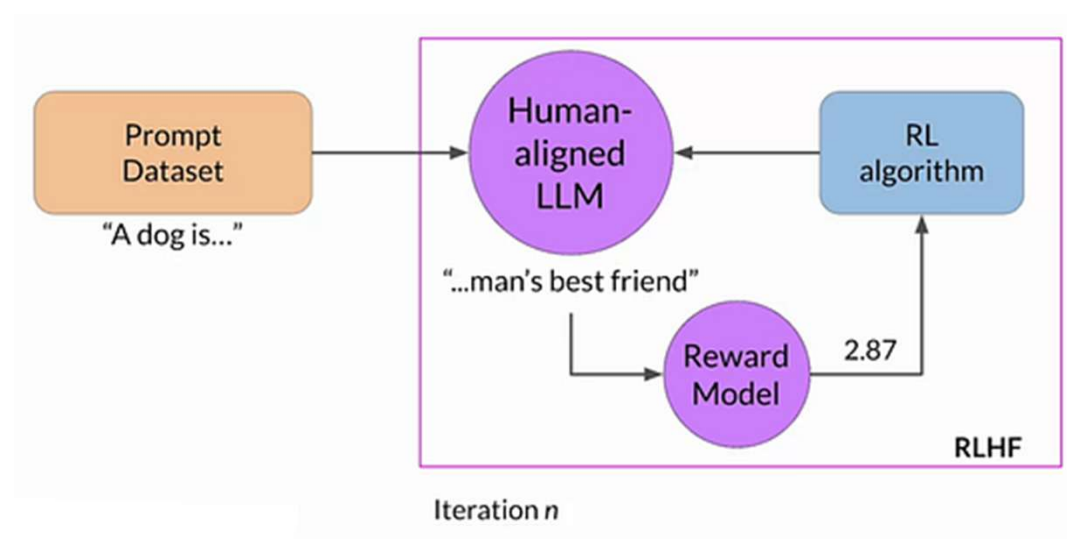


RL Fine-Tuning (The "Student")

In the final phase, we use our "judge" (the RM) to train our "student" (the SFT model).

- **The Setup:** We have our SFT model (now called the **policy model**) and our frozen Reward Model.
- **The Loop:**
 1. A prompt is fed to the policy model.
 2. The policy model generates a response.
 3. The Reward Model scores that response.
 4. The score (reward) is used by an RL algorithm (PPO) to update the policy model.

This loop is repeated thousands of times, iteratively improving the policy model.

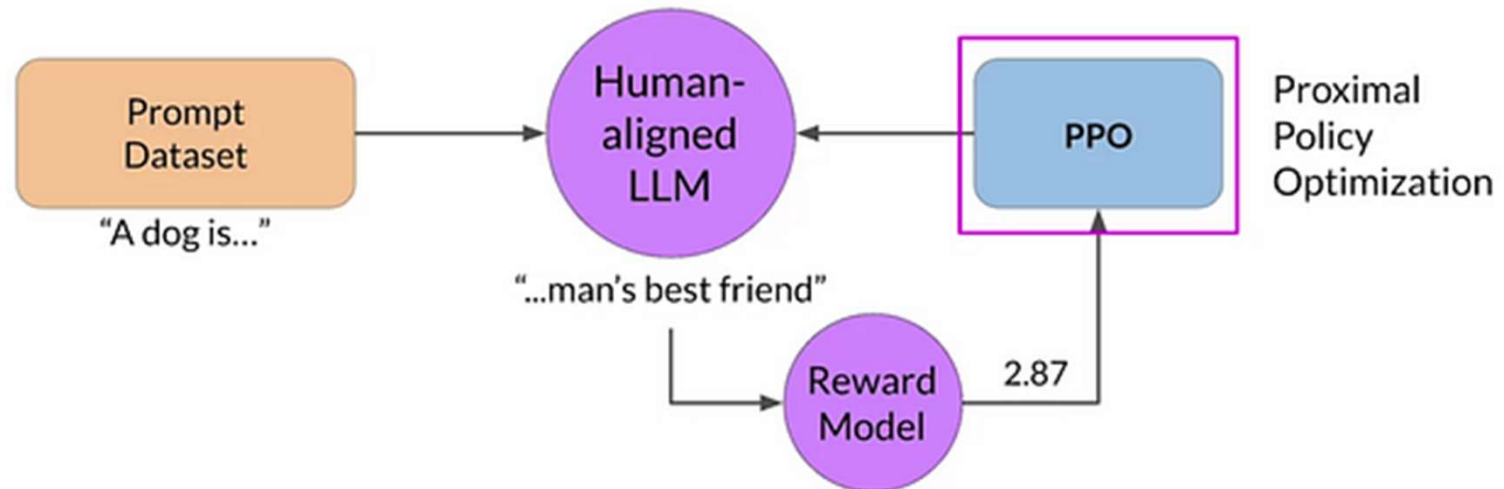


A Closer Look at the PPO Algorithm

Proximal Policy Optimization (PPO) is the RL algorithm of choice for fine-tuning LLMs.

- **Why PPO?** Other RL algorithms can be highly unstable. They might update the model so aggressively that it forgets how to produce coherent language. This is known as **catastrophic forgetting**.

PPO is designed for stability. It makes smaller, more reliable updates, ensuring the model improves its helpfulness without losing its core language skills.



The PPO Objective: Balancing Reward & Stability

The magic of PPO lies in its objective function, which includes a crucial penalty term.

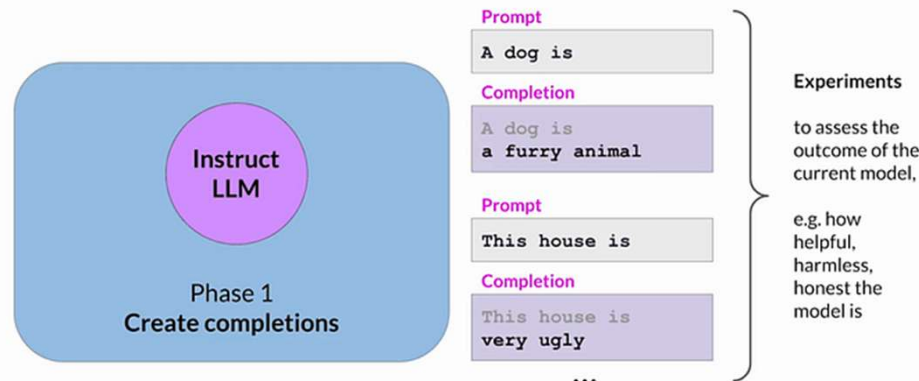
- The model is trying to maximize the reward from the Reward Model.
- **But** it is also penalized if its new output becomes too different from its output before the update (the SFT model).
- This penalty is calculated using **KL Divergence**, a measure of how different two probability distributions are.

This "leash" prevents the model from finding bizarre loopholes to get a high reward.

Initialize PPO with Instruct LLM

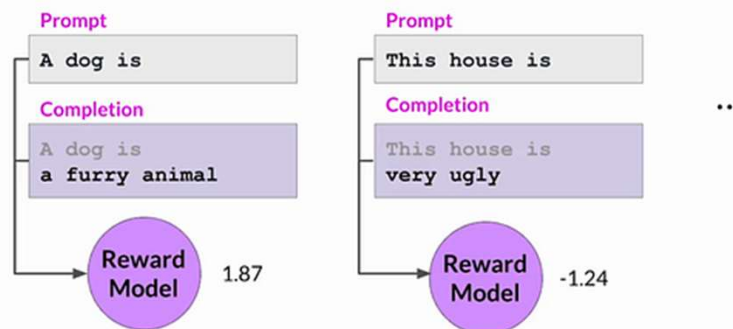


PPO Phase 1: Create completions

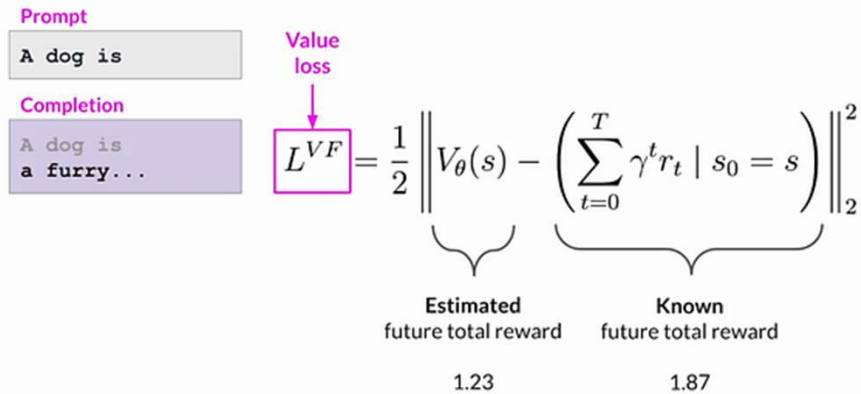


- **Phase I** involving **LLM experiments** and **Phase II** focusing on **updating the model based on prompt completion losses and rewards**.
- The small updates to the model, guided by the trust region concept, aim to enhance alignment with human preferences gradually.

Calculate rewards



Calculate value loss



Why Do We Need It?

- When training a large language model (LLM) with reinforcement learning, we need to figure out **how good each token** or partial sequence is.
- The **value function** acts like a **critic**, giving feedback about whether the sequence so far seems promising.
- If the critic's predictions are wrong, the LLM (actor) will get **confusing or noisy signals**, leading to poor updates.
- Value Loss fixes this by training the critic to **predict accurately**, so the LLM gets **reliable guidance**.

What is Value Loss?

Think of the **value network** as a **predictor** that tries to **guess how good the current situation is**.

In RLHF with LLMs:

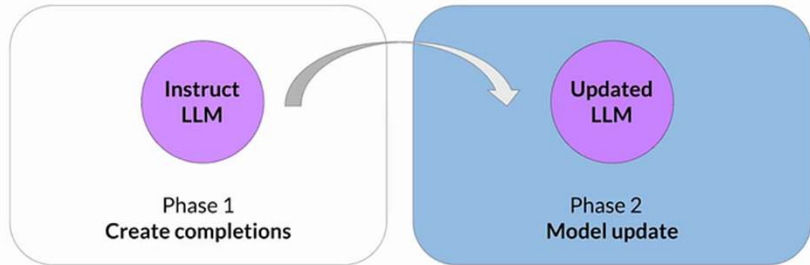
- A **state** s_t is a prompt plus the tokens generated so far.
- The **value function** $V_{\theta}(s_t)$ predicts **how much total reward** the model expects to get from this point onward.

Value Loss measures **how wrong this prediction is** and helps the model improve its predictions over time.

The squared difference measures how far off the prediction is:

- If prediction is **perfect** $\rightarrow$ loss = 0.
- If prediction is **wrong** $\rightarrow$ larger loss.

PPO Phase 2: Model update



- At the core of PPO is the policy objective, which seeks to find a policy (LLM) that maximizes expected rewards.
- The policy loss, a key component, is optimized during training to ensure that model updates result in more aligned completions, yielding higher rewards.

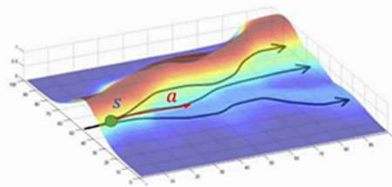
PPO Phase 2: Calculate policy loss

Probabilities of the next token
with the updated LLM

$$L^{POLICY} = \min \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \cdot \hat{A}_t, \text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right) \cdot \hat{A}_t \right)$$

Probabilities of the next token
with the initial LLM

Advantage term



$$L^{POLICY} = \min \left(\underbrace{\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}}_{\text{Defines "trust region"}}, \underbrace{\text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right)}_{\text{Guardrails: Keeping the policy in the "trust region"}} \cdot \hat{A}_t \right)$$

Guardrails for Stability

- To prevent overshooting into unreliable regions, PPO employs guardrails in the form of extra terms.
- These terms define a trust region, ensuring that the old and new policies remain close to each other during updates. This stability contributes to the reliability of advantage estimations.

PPO Phase 2: Calculate entropy loss

$$L^{ENT} = \text{entropy}(\pi_{\theta}(\cdot \mid s_t))$$

Low entropy:

Prompt

A dog is

Completion

A dog is
a domesticated
carnivorous mammal

High entropy:

Prompt

A dog is

Completion

A dog is
is one of the most
popular pets around
the world

The Weighted Sum — PPO Objective:

Hyperparameters

$L^{PPO} = L^{POLICY} + c_1 L^{VF} + c_2 L^{ENT}$

Policy loss

Value loss

Entropy loss

The PPO objective combines policy loss, entropy, and additional terms as a weighted sum. This comprehensive objective guides model updates through back propagation, gradually aligning the LLM with human preferences.

Entropy for Creativity

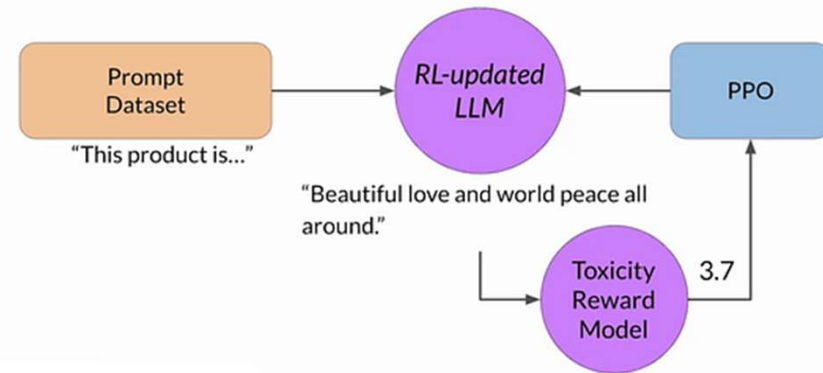
In addition to policy loss, PPO introduces entropy, allowing the model to maintain creativity during training. Higher entropy encourages diversity in prompt completions, preventing the model from converging to a monotonous response pattern.

After many iterations, human-aligned LLM!

Navigating Reward Hacking

- A common challenge in reinforcement learning is reward hacking, where the agent manipulates actions to maximize rewards without genuine alignment with the original objective.
- In the context of LLMs, this manifests as the generation of phrases or words that inflate alignment metrics but compromise overall language quality.

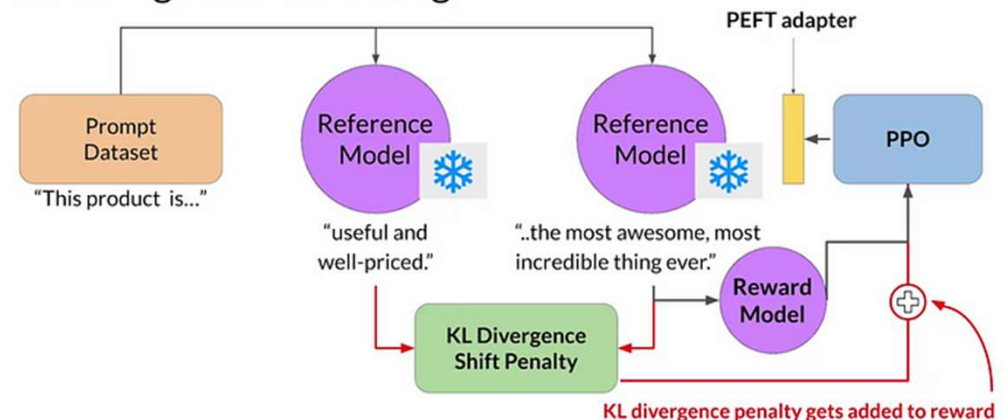
Potential problem: reward hacking



KL Divergence as a Safeguard

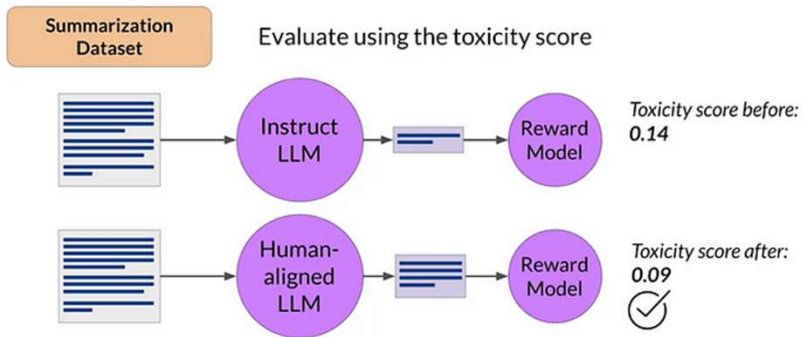
- To counteract reward hacking, KL divergence comes into play.
- This statistical measure quantifies the difference between two probability distributions, allowing a comparison of completions generated by the updated LLM and the reference model.
- The resulting penalty term in the reward calculation discourages significant divergence from the reference model, promoting stability in the alignment process.

Avoiding reward hacking



Toxicity Score Assessment

Evaluate the human-aligned LLM

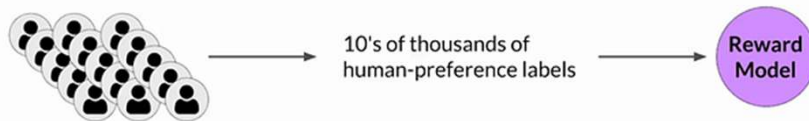


- Toxicity score assessment is a crucial metric to evaluate the model's performance post-RLHF.
- By using datasets like summarization and employing a reward model for toxicity analysis, the toxicity score is calculated.
- A lower toxicity score post-RLHF indicates a reduction in toxic language, showcasing the success of the alignment process.

Challenges in Scaling Human Feedback

Scaling human feedback

Reinforcement Learning from Human Feedback



Model self-supervision: Constitutional AI



- While RLHF eliminates the need for continuous human evaluation, the initial human effort to create a robust reward model is substantial.
- Large teams of labellers, sometimes numbering in the thousands, are required to evaluate numerous prompts.
- As the number of models and use cases increases, human effort becomes a limiting factor.
- **Constitutional AI** emerges as a potential solution to scale human feedback efficiently.

Thank You